

TECHNICAL SPECIFICATION DOCUMENT (TSD)

Project Name: ESP Flasher Auto

Release Version: 5.0

Document Type: Software Architecture & Technical Reference

Target Audience: DevOps, IT Infrastructure, Production Engineers

1. SYSTEM OVERVIEW

O ESP Flasher v5.0 é uma aplicação que orquestra a gravação de firmware em *microcontroladores* (famílias ESP8266 e ESP32), captura de telemetria de hardware (MAC, IMEI, CIMI) via interface UART, e geração assíncrona de etiquetas de rastreabilidade utilizando comandos raw ZPL via protocolo TCP/IP.

2. SYSTEM ARCHITECTURE

A arquitetura baseia-se num modelo cliente-servidor local (*localhost*), operando de forma assíncrona para garantir a não-obstrução da interface (UI) durante processos de I/O bloqueantes (operações serial e de rede).

2.1. Backend Service (Daemon)

- **Runtime:** Python 3.8+
- **Web Server:** Flask atuando como WSGI handler.
- **IPC (Inter-Process Communication):** Utiliza Flask-SocketIO acoplado ao motor eventlet/threading para streaming de *stdout* (buffer serial) para o DOM do browser.
- **Concurrency:** As rotinas de *flashing* (esptool) e o *listener* da porta COM são executados em *threads* separadas (Daemon Threads) libertando o *Main Thread* do servidor web.

2.2. Frontend Client

- **Stack:** HTML5, Bootstrap 5, Vanilla JS.
- **Event Handling:** Socket.IO Client (v4.0.1) para *event listeners* bidirecionais.
- **DOM Updates:** Modificação direta do DOM em tempo real baseada nos *payloads* emitidos pelo motor de *parsing* do backend.

3. HARDWARE INTERFACING & SERIAL PIPELINE

3.1. Flashing Engine

A gravação é delegada ao `esptool.py` através do módulo `subprocess` do SO.

- **Standard Args:** `--before default-reset --after hard-reset write-flash --flash-mode dio 0x0`
- **Velocidades Suportadas:** 9600, 115200, 460800, 921600, 1.5M, 2.0M baud.

3.2. Serial Monitor & Data Buffer

A thread de monitorização cria uma instância `serial.Serial(port, baudrate, timeout=1)`. O buffer é lido através de `readline().decode('utf-8', errors='ignore')`.

- **Buffer Management:** O buffer de string global mantém apenas os últimos 10.000 caracteres em memória (`buffer_serial[-10000:]`) para prevenir *memory leaks* (OOM) em execuções prolongadas (sessões >24h).

4. DATA PARSING ENGINE (REGEX)

A captura de telemetria é acionada via API POST (`/analisar_buffer`) ou através do *Stream Listener*. O motor de expressões regulares (Regex) varre o buffer à procura de padrões industriais normalizados:

1. MAC Address Resolution:

- Padrão Standard: `r'([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})'`
- Padrão Flat: `r'mac:\s*([0-9A-Fa-f]{12})'`
- *Output Layer:* Conversão automática em tempo de execução para formatos derivados: Flat Lowercase (QR Code), Clean (CSV).

2. IMEI & CIMI GSM Extraction:

Os módulos celulares injetam as respostas AT no buffer (e.g., AT+CIMI).

- O motor procura o prefixo de comando explícito: `r'AT\+CIMI[\s\S]*?<<\s*\d{15})'`
- Caso não localize os *headers* AT, ativa o *fallback scan* para instâncias isoladas de 15 dígitos: `r'\b\d{15}\b'`.

3. Serial Number Generation:

Se não for providenciado *input* manual no DOM, é injetado um *pseudo-random integer* de 10 dígitos (via módulo `random`).

5. PRINT SUBSYSTEM (ZPL INJECTION)

O sistema ignora o Spooler de Impressão do SO (Windows Print Spooler / CUPS) para minimizar a latência (< 50ms per job).

5.1. Macro Replacement Protocol

Ficheiros Zebra (.lbl, .zpl) são lidos em memória (bytes, convertidos para utf-8 ou iso-8859-1).

Ocorre um mapeamento e injeção (.replace()) das seguintes variáveis reservadas:

- @MAC@ -> Original (28:05:A5:1F:71:70)
- @MAC_CLEAN@ -> *Stripped* (2805A51F7170)
- @MAC_LOWER@ -> *Lowercase stripped* (28:05:a5:1f:71:70) - Usado tipicamente dentro do subset de ^BQN (QR Code).
- @IMEI@, @CIMI@, @SN@ -> Mapeamento direto de valor numérico.

5.2. TCP/IP Socket Transmission

O código RAW pós-processado é disparado para a interface de rede:

- **Protocolo:** TCP IPv4.
- **Módulo:** socket.AF_INET, socket.SOCK_STREAM.
- **Porta:** 9100 (RAW TCP / HP JetDirect padrão).
- **Timeout:** 3.0 segundos. Falhas disparam a exceção [Errno 11001] getaddrinfo failed devolvida à API REST.

6. DATA PERSISTENCE LAYER

1. Production Log (CSV)

- **Local:** dispositivos_registados.csv.
- **Regra de Escrita:** Appends de linha ('a'). Aplica lock de I/O momentâneo (os.fsync) após o *flush* para evitar corrupção de ficheiro em falhas de energia do host.

2. Hardware Profiles (JSON)

- Ficheiro estruturado perfis.json armazenando um Array de *Objects* com os campos id, nome, file e baud.

7. BUILD CHAIN & PACKAGING

A aplicação não necessita de containerização (Docker).

- **Toolchain:** PyInstaller (gerado através de auto-py-to-exe).
- **Format:** Monolithic Standalone Executable (--onefile no Windows, Binário no Linux).
- **Bundle Config:**
 - O diretório templates/ é agregado à raiz temporal (_MEIPASS) através da flag --add-data.
 - Injeção forçada do *import* oculto --hidden-import "engineio.async_drivers.threading" vital para o *loop* assíncrono do Eventlet no executável compilado.
- **Boot Sequence:** O executável auto-extrai as bibliotecas e o interpretador Python embutido para uma pasta temporal (%TEMP% no Windows) e emite a chamada HTTP ao Chromium/Edge do sistema para apresentar a interface GUI.